课程笔记：监督微调（SFT for Agents）

Bowen-mantou（整理）

2026-10-23

视频作者/频道：B 站「常青藤名校课」· CMU 11-768 AI Agents（英文原声字幕版）
发布日期：2026-09-09
视频时长：64:59
视频链接：https://www.bilibili.com/video/BV1a6Yx62EH4?p=22
PDF 制作 Skill：https://github.com/wdkns/wdkns-skills

目录

1 引言：为什么单独讲「训练」 2

1.1 更新 agent 的三条路 . . . 2
1.2 从一条 SWE 轨迹看「训练要学什么」 . . . 2
1.3 本章小结 . . . 2

2 SFT 在训练流水线中的位置：冷启动 3

2.1 预训练、中期训练、SFT、RL . . . 3
2.2 为什么不能跳过 SFT 直接上 RL . . . 3
2.3 SFT 与 RL 的对照 . . . 3
2.4 本章小结 . . . 3

3 轨迹作为 token 序列：模板、mask 与损失 4

3.1 模板把整条轨迹渲染成一个字符串 . . . 4
3.2 只让 assistant token 承担损失 . . . 4
3.3 mask 是怎么被构建出来的 . . . 4
3.4 主流框架的 mask 开关与 end-of-turn . . . 4
3.5 reasoning block：训还是不训 . . . 4
3.6 样本权重不等于 token 权重 . . . 5
3.7 这一块的开放问题 . . . 5
3.8 本章小结 . . . 5

4 选轨迹：从哪里来、怎么过滤 5

4.1 轨迹来源的四象限 . . . 5
4.2 成功轨迹里也可能有坏行为 . . . 6
4.3 按形状过滤：最小轮数 . . . 6
4.4 选老师：最强不等于最好教 . . . 6
4.5 任务来源与数据多样性 . . . 6
4.6 课堂讨论：三条轨迹怎么选 . . . 6
4.7 本章小结 . . . 7

5 统一格式与多样性：Agent Data Protocol 7

5.1 异构数据集带来的工程墙 . . . 7
5.2 中间格式：每个数据集只转一次 . . . 7
5.3 多源混合确实比单源好 . . . 7
5.4 本章小结 . . . 7

6 跑通一次 SFT：工程细节 7

6.1 来自真实技术报告的配置 . . . 7
6.2 MoE 的专家路由 . . . 8
6.3 打包整段对话，不截断 . . . 8
6.4 模板漂移（template drift） . . . 8
6.5 公开的训练工件 . . . 8
6.6 本章小结 . . . 8

7 训练好不等于评测好：训练与部署的失配 **9**
7.1 训练只走记录过的动作 9
7.2 评测的可信度清单 9
7.3 本章小结 9

8 SFT 与 RL 的交接：SFT 的收益在哪里 **9**
8.1 三条经验 9
8.2 没有更强的教师怎么办 10
8.3 要不要监督错误答案 10
8.4 本章小结 10

9 总结与延伸 **10**

1 引言：为什么单独讲「训练」

这一讲是第一讲关于**训练（training）**的课：前四讲讨论的都是「不改权重、只在推理时做文章」的 agent 增强（上下文、工具、记忆、技能），而从这一讲开始，我们要把学到的经验写进**模型权重**，并在接下来的三讲里依次展开 SFT、强化学习基础与深度研究。

1.1 更新 agent 的三条路

课堂开场先回顾了一个贯穿全课的三分法——想让 agent 变强，可以更新三种东西：

载体	优点	代价
上下文窗口	保真度高、改动即时生效	贵、噪声大、不决定「什么重要」
外部工件	可检查、可编辑、可检索、可移植	需要诱导、选择与持续维护
模型权重（本讲）	推理更快、行为改变更彻底，一次训练后每次调用都带着新行为	训练慢、过程不透明、与具体模型绑定、单向不可逆

一句话概括三者的时间尺度：上下文与外部工件可以在「秒」级更新，而改权重每次都要付一次训练成本；权重还不能打开看内部发生了什么，也不能把这次改变原样搬到另一个模型上。因此训练这条路换来的是**快推理**、**广覆盖**、**强行为改变**，代价是**慢**、**黑盒**、**模型专属**。

1.2 从一条 SWE 轨迹看「训练要学什么」

课堂用一条软件工程（SWE）轨迹做例子：agent 收到一个 issue、拿到一个沙箱里的若干工具（例如 Bash、文件编辑），它一边产生思考、一边发出命令、读取命令返回的 observation，最终由仓库自带的单元测试判定这次运行是否成功。一条轨迹里有四种角色：system、user、assistant，以及 observation；而真正由模型产生的只有 assistant 消息。

那么一条轨迹究竟能教模型什么？课堂列了四类：第一，**如何写出可被 harness 执行的工具调用**；第二，**如何在对话变长时继续下去**（因为训练数据里的轨迹本来就是长的）；第三，**工具结果应该长什么样**——模型要学会「等一个结果」，而不是自己把结果也写出来；第四，数据里附带的其他东西，比如额外的解说、重复的命令，以及在多种可行解法里被选中的那一种。这最后一点是关键：SFT 会把这些「数据里恰好出现的习惯」一起学走，不管它们好不好。

1.3 本章小结

改 agent 有三条路：上下文、外部工件、权重。本讲专攻权重：它换来快推理与彻底的行为改变，代价是训练成本、黑盒性和模型绑定。一条轨迹能教工具调用格式、长对话续写、观测格式，也会连数据里的坏习惯一起教——这决定了后面「选数据、过滤数据」的重要性。

2 SFT 在训练流水线中的位置：冷启动

2.1 预训练、中期训练、SFT、RL

课堂给出一张完整训练流水线的图：**预训练**（pre-training）用最多 token 让模型学会下一 token 预测；**中期训练**（mid-training）是预训练与后训练之间的一切（定义各家不一，甚至有人把 SFT 也算进去）；**SFT** 与 **RL** 属于后训练（post-training）。一个关键区别是：预训练、中期训练、SFT 都能找到**固定语料**来训练，而 RL 通常没有固定语料，它训练的是模型**自己的 rollout**。

课堂以 Kimi K3 的报告为参照给出了一种常见后训练配方：先做 SFT，**初始化基线的 agent 能力**；再做 RL，**训练不同推理强度的领域专家**；最后做 **on-policy distillation**，把专家合并回一个模型。这里的 SFT 就是所谓「冷启动（cold start）」：它决定 RL 从哪个 checkpoint 出发。课堂强调：在 Kimi K3 中，模型是在**任何 RL 之前**的 SFT 阶段就学会了调用工具、完成长任务。

2.2 为什么不能跳过 SFT 直接上 RL

一个自然的反问是：既然 RL 更强，为什么不从 base model 直接 RL? 课堂给了 DeepSeek R1.0 的反例：R1.0 在 base model 上直接做 RL、完全跳过 SFT，结果模型的回答**中英混杂、难以阅读**；在紧接着的下一代模型里，他们又把 SFT 加了回来——R1 从几千条精心挑选的 SFT 样例出发。课堂的概括是：**SFT 阶段为后续 RL 建立了一个高质量的冷启动策略**。

SFT 的定位

SFT 不是「比 RL 弱的替代品」，而是 RL 的**起点**。它负责把通用 agent 能力（工具调用、格式、长任务）装进模型，RL 负责在此基础上把某个领域的能力磨尖。

2.3 SFT 与 RL 的对照

课堂把两者的差异压成一张表：

	SFT	RL
学习来源	记录下来的轨迹（不论由谁产生）	模型自己的 rollout，由奖励打分
信号	每个 token 一个目标	每条 rollout 一个数
需要	数据 + GPU	活的环境、沙箱、可检查的奖励
擅长	学工具调用、格式、长任务	把模型已经会尝试的东西磨尖
失效	示范本身错误、狭窄或已耗尽	起点太弱，或奖励不可检查

2.4 本章小结

训练流水线是「预训练 → 中期训练 → SFT → RL」，SFT 是 RL 的冷启动：先装通用 agent 能力，再由 RL 磨领域能力。DeepSeek R1.0 的教训说明跳过 SFT 直接 RL 不可读、不可用；SFT 与 RL 的信号、依赖和失效边界完全不同，二者是接力关系而非替代关系。

3 轨迹作为 token 序列：模板、mask 与损失

3.1 模板把整条轨迹渲染成一个字符串

语言模型训练只认 token 序列，因此要先把一条对话轨迹用 **chat template** 渲染成一个字符串：system、user、assistant、observation 之间的边界由模板的特殊 token 标出（例如 Qwen 系列的 `<|im_start|>`、`<|im_end|>`、tool call 标记等）。一条轨迹里可能有十几次工具调用、最后提交给 verifier 的最终答案，但训练基础设施看到的只是**一整个字符串**。

3.2 只让 assistant token 承担损失

与普通语言模型 SFT 不同，agentic SFT 通常**不训练整条轨迹**：系统提示、用户消息、工具返回的 observation 是「上下文」，不算损失；只有 assistant 自己写出来的部分才算预测目标。具体做法是给每个 token 配一个 0/1 的 mask，损失写成

$$\mathcal{L}(\theta) = -\sum_{t:\, m_t=1} \log p_\theta(x_t \mid x_{<t}),$$

其中 x_t 是第 t 个 token，$x_{<t}$ 是它之前的全部上下文，p_θ 是模型给出的条件概率，$m_t \in \{0,1\}$ 是 mask 位：1 表示这个 token 参与损失，0 表示只作为上下文。最小化它，就是在提高「给定前文时复现记录动作」的概率——这正是行为克隆（imitation learning）的目标。

为什么不在 observation 上训练？课堂的解释是：观测由用户或环境给出，不是 assistant 产生的，训练它等于让模型去预测「别人会说什么」。不过课堂也补充：确实有工作训练观测以帮助模型学习世界状态，所以这是「一般配方」，值得按场景做消融。

3.3 mask 是怎么被构建出来的

mask 的构建分三步：**渲染**（renderer 用模板把对话写成整串并记下每个 assistant 段的起止）、**分词**（字符位置变成 token 位置）、**掩码**（assistant 段内为 1，其余为 0）。这是「模板做记账」的直接后果：训练开始时，system/user/observation 这些角色信息已经消失了，训练器只看到一个字符串和一串 bit。

3.4 主流框架的 mask 开关与 end-of-turn

课堂点名了几个 SFT 框架的掩码控制：TRL、Axolotl、LLaMA-Factory 都提供「是否训练 prompt」「是否 mask 历史」「是否训练整条轨迹」之类的开关。实操中最常见的错误是**分不清哪些该算损失、哪些只是上下文**：例如 assistant 消息的段头（row header）不是模型生成的，不该训练；模型真正生成的是内容段；最后还有一个停止 token 要训练；而 **end-of-turn 往往由多个事件组合**——结束消息、结束工具调用、结束任务。Kimi K3 的模板把它们拆成独立 token，让模型更容易学会「在哪里停」。

3.5 reasoning block：训还是不训

另一个重要开关是 **reasoning（思维链）块**。reasoning 轨迹通常远长于真正的工具调用，若全部计入损失，损失会被长推理主导，模型可能学会「只推理、不动手」。常见做法有三种：把 reasoning 留在上下文里但不训练（它解释后面的动作）；训练它（模型因此学会自己写这种风格）；或者干脆把它移出上下文。课堂举了 Nemotron 3 Ultra 的例子：它把被预算截断的 reasoning 留在上下文里，

但把人为截断的那一段 mask 掉、不计损失。另有工作（Balanced SFT）专门**平衡 reasoning 与工具调用**两类轨迹，避免一类压过另一类。

三个常见的 mask 错误

1. 把 assistant 段头或模板特殊 token 当成可训练 token；
2. 忘记训练 end-of-turn / stop token，导致模型不会停、把对话一直写下去；
3. 让超长的 reasoning 主导损失，训练出一个「只会想、不会做」的 agent。

3.6 样本权重不等于 token 权重

课堂用 MAI（Microsoft AI）的 Consolidation SFT 配方说明一个容易忽略的偏差：他们的数据混合**按样本数**配比为 STEM 与 Coding 占 56%、Agentic Capability 占 11%、General Helpfulness 与 Safety 占 33%；但损失是**按 token** 计算的，由于 STEM/Coding 轨迹特别长，最终 token 占比变成 89% / 9% / 2%。

配比检查清单

如果你按「轨迹条数」平衡数据，还要再检查一次「token 分布」是不是你想要的；反过来，按 token 平衡也可能让短而重要的能力被稀释。两种平衡都只是手段，最终要看评测。

3.7 这一块的开放问题

课堂明确指出两点：其一，**没有公开实验**系统改变 agent SFT 的 mask 并测量其影响；其二，关于「是否训练 prompt」的谨慎实验大多发生在**单轮指令微调**上，而 agent 轨迹恰好相反（长 prompt、短答案、数据少）。因此「该不该在 observation 或 prompt 上训练」在 agent 场景仍是开放问题。

3.8 本章小结

一条轨迹先被模板渲染成一个 token 字符串，再用 0/1 mask 只让 assistant token 进损失，这就是行为克隆。mask 由「渲染 → 分词 → 掩码」三步构建，框架给了各种开关；end-of-turn 与 reasoning 块的处理会显著改变学到的行为；配比要同时看样本与 token。这些细节没有统一最优解，需要按场景做消融。

4 选轨迹：从哪里来、怎么过滤

4.1 轨迹来源的四象限

课堂把 agent SFT 数据的来源总结成 2×2：谁跑的任务（更强的模型 / 模型自己）× 保留全部还是只保留通过验证的轨迹。只保留强模型生成的通过轨迹是**最常见**的一格（SWE-Gym、Nemotron、OT-Agent 等都在此列）；全部保留叫 plain distillation；模型自己生成且全部保留，即 MAI 的自蒸馏；模型自己生成但只保留通过验证的，就是 **expert iteration**（后续课程会讲）。

课堂以 SWE-Gym 的结果说明强模型蒸馏的收益：用 GPT-4o、Claude 等强模型跑轨迹、只保留通过仓库测试的轨迹，学生模型的解决率显著提高。

4.2 成功轨迹里也可能有坏行为

一个重要提醒：即使最终任务成功，中间也可能包含不该学的东西。课堂展示了两条「通过了单元测试」的轨迹：一条大部分回合都在「改一下、重跑测试」的循环里空转；另一条的最终补丁里留着调试用的 print 语句。SFT 的损失会**照抄每一个动作**，好坏一起学——于是「会修」和「会乱来」被同时刻进权重。

4.3 按形状过滤：最小轮数

最朴素的过滤是**按轨迹长度/轮数**。课堂引用了 openhands agent 论文的消融：在他们的设置里，最好的单一过滤是**把少于 5 轮的轨迹全部丢掉**。理由很直接：轨迹太短，要么是一把就做对的简单题（学不到错误恢复），要么是生成过程本身就出了问题。

课堂也诚实指出这种过滤的盲点：轮数只衡量「花了多少功夫」，不衡量「功夫好不好」。一个 4 轮干净修复会被丢掉，一个 14 轮乱七八糟的轨迹反而会被保留——但平均而言它仍然有帮助，这就是工程上的「够用的启发式」。

4.4 选老师：最强不等于最好教

课堂用一张教师模型消融表说明：把同一个学生模型分别蒸馏自不同教师，教师的「自身强弱」与「教得好不好」并不一致。表中最强的 GPT-5.3-Codex 反而是最差的老师（平均 11.94、归一化 −1.47），比量化后的 GLM 4.7（18.16、+0.73）在 Terminal-Bench 2.0 上还低约 5 个百分点；Kimi K2.5 与 GLM 5 居中。

可能的原因：强模型的「思考方式」与学生差异太大，轨迹对学生不可学；排行榜只说明模型自己会做题。工程建议是**多试几个老师**，并用代价低的小模型快速验证学生收益。

4.5 任务来源与数据多样性

课堂强调：不同任务来源对不同 benchmark 的贡献差异很大。例如 SWE-smith 对 SWE-bench Verified 贡献最大，但对 Terminal-Bench 贡献较小——**任务源的选择可能比其它任何超参都更影响结果**。经验结论是：混合 **4–8 个来源**最好，铺得更宽反而不再有帮助。

另一个反直觉发现是「数据越多越好」并不总成立：如果只是从同一分布**上采样**（同一任务的更多 rollout），性能会饱和甚至下降；而增加**分布多样性**（新任务、合成增广）时，性能还能继续上升。课堂用三张曲线对比了「同分布上采样」与「合成增广」。

4.6 课堂讨论：三条轨迹怎么选

课堂给了一个讨论题：同一个任务有三条轨迹——A 是 4 轮就干净修好的短轨迹；B 是长轨迹，中间做错了一次编辑、测试失败后又恢复；C 是试了 11 次、每次都没改对，最后却解决了。该保留哪几条、哪些动作该计损失、最小轮数过滤在这题上会错在哪？

课堂的结论是 **A 和 B 都要**：B 教的是错误恢复能力（这正是 agent 最稀缺的能力之一），A 教的是干净利落；但 B 也有风险——模型可能只学会「先做错再改」，所以要在数据里保留两端、并控制比例。而「最小 5 轮」的过滤在这题上会**丢掉 A、留下 C**，恰好展示了启发式过滤的代价。

4.7 本章小结

轨迹来源要看清「谁生成、是否过滤」；成功轨迹也可能带坏行为，需要按形状/规则过滤；教师不是越强越好，要实测学生收益；任务源与数据多样性比单纯堆数据重要（4–8 个来源、增广优于上采样）；课堂讨论说明「保留正确 + 恢复两类轨迹」比机械按轮数过滤更接近想要的 agent 能力。

5 统一格式与多样性：Agent Data Protocol

5.1 异构数据集带来的工程墙

课堂介绍了主讲人自己的一个工作（与 Graham Neubig 教授合作）：**Agent Data Protocol**。动机很实际——现存的 agentic SFT 数据集由不同团队、针对不同目标、用不同格式构建：同样做 web 任务，有的用 HTML 作为观测，有的用 accessibility tree；工具调用的表示也各不相同。如果想把 GoBrowse 与 Mind2Web 一起训练，就得为每个数据集写一份到目标 agent 格式的转换器；换成 OpenHands，又要重写一遍。数据集一多，转换器数量是「数据集 × 目标 agent」的乘积。

5.2 中间格式：每个数据集只转一次

Agent Data Protocol 的思路是引入一个**统一的标准格式**：每个数据集只需要转换一次到标准格式，再从标准格式转换一次到目标 agent 格式。web 数据集中，HTML 与 accessibility tree 两种观测都保留在统一结构里，下游可以自选；这样既减少了工程量，又让多数据集混训成为可能。转换之后仍需检查：tool call 是否都能解析、每个调用是否带 function schema、对话是否按预期结束。

5.3 多源混合确实比单源好

论文做了对照实验：固定同一个 harness、同一个模型、同一套评测，只改变训练数据的混合。结论是：**几乎每个 agent benchmark 上，多源混合都优于单源训练**，甚至在该领域自己的 benchmark 上，「混合」也能打败「恰好匹配该领域的单源数据」。这说明数据多样性带来的收益大于「领域精确匹配」的收益。

5.4 本章小结

Agent 数据格式高度异构，逐对写转换器不可扩展；统一中间格式把「数据集 × agent」的乘积降为「数据集 + agent」两段线性转换。多源混训在几乎所有 benchmark 上优于单源，多样性是 agent SFT 的重要杠杆。

6 跑通一次 SFT：工程细节

6.1 来自真实技术报告的配置

课堂列了几份真实配置作参照：Nemotron 3 Ultra 把序列打包到约 294K token、batch size 64，学习率随时间衰减；MAI 的自蒸馏用 128K 序列、很大的 batch，并在正式 SFT 前做 warm-up；另一个近期开源模型用 512K 序列、分几个阶段，学习率在高量子集上衰减。课堂的建议是：动手前至少读一两份前沿实验室的技术报告，里面有很多可以借鉴的细节。

6.2 MoE 的专家路由

前沿模型大多是 MoE（混合专家）。课堂指出一个现象：在狭窄的 agent 数据上微调 MoE 时，**少数专家会吃掉大部分 token**（图右侧：专家序号越大，分到的 token 越少）。MAI 在 SFT 阶段对路由做很强的平衡，难度比 RL 阶段高约一千倍，并把 dropout 提到 0.15。

为什么 SFT 阶段的路由更容易失衡

SFT 数据窄、格式单一，路由器会迅速把同类 token 都送给同一个专家，形成「赢者通吃」。一旦路由塌缩，模型容量利用率下降，后续 RL 也会被限制。因此 MoE 的 SFT 往往需要辅助的负载均衡损失或路由约束。

6.3 打包整段对话，不截断

打包（packing）的常见做法是把多条对话塞进固定长度的序列里以提高利用率，但 agent SFT 有一个额外约束：**不要截断、不要切分轨迹**，要保留完整对话。课堂给出的做法是：每条对话被分配到「剩余容量最贴合它」的那个 pack，任何对话都不被切开；相同问题的对话不放进同一个 pack，pack 之间再随机打乱。这样做的一个动机是**避免幻觉**——被切断的轨迹会让模型学到错误的续写边界。

6.4 模板漂移（template drift）

一个容易被忽视却致命的工程细节是：**训练用的模板必须与部署用的模板完全一致**，甚至空白字符都要一致。课堂给了一个具体场景：同一条轨迹，在「工具消息已经追加」前后渲染两次，会得到两个不同的字符串；一旦条件性的 thinking block 改变了前面 assistant 段的结尾，前缀就不再匹配。在**工具循环**里前缀不匹配是致命的——模型在训练时见到的上下文与推理时见到的不是同一个东西，行为会不可预期地退化。

模板漂移是「静默失败」

它不会让 loss 报错，只会让训练出来的能力在部署时打折。上线前务必用同一个 renderer 分别渲染「训练样本」和「真实推理请求」，逐 token 对比。

6.5 公开的训练工件

课堂推荐关注公开发布训练细节的模型：有的 model card 给出分阶段训练表（每阶段的 token 数、序列长度、目的），公开 W&B 上的训练损失曲线，甚至放出中间 checkpoint 供继续训练或后训练；也有模型同时发布后训练 checkpoint、训练数据与配方。对想学习「工业级 SFT 长什么样」的人来说，这些工件比论文正文更有信息量。

6.6 本章小结

真实 SFT 的工程点集中在：序列打包不截断、MoE 路由平衡、模板与部署严格一致、学习率随阶段衰减。技术报告与公开工件是学这些细节的最佳材料；其中模板漂移是最容易静默毁掉一次训练的坑。

7 训练好不等于评测好：训练与部署的失配

7.1 训练只走记录过的动作

训练与推理有一个结构性差异：训练时，模型在**记录的轨迹**上一步一步算损失，每一步的上下文都是真实发生过的；而部署时**没有记录**，模型每做一个动作就会改变接下来能看到的状态。课堂的例子是：模型做了一次与训练数据不同的编辑，测试失败，于是进入一个「训练数据从未访问过的状态」；此时怎么做，取决于它的错误恢复能力，而这正是 SFT 数据里可能没有的。

因此课堂给了一个重要判断：**训练损失曲线可以用来监控训练是否正常，但不能替代真实评测**。低损失只保证「在记录过的状态上能复现动作」，不保证「在没见过的状态上做对」。

7.2 评测的可信度清单

课堂给了一串很实用的评测建议：

1. **先过拟合 20 个样本**：如果 loss 降不下去，bug 大概率在上游（数据、模板或 mask）；
2. **用 harness 跑有可检查结果的任务**，而不是只看训练 loss；
3. **小心读论文数字**：有些数字是「多个 harness 里最好的那个」，不等于你单次单 harness 能复现的结果；
4. **让 split 匹配你要主张的结论**：要声称「泛化到新仓库」，就要按 repository 切分，并保证同一任务的所有 run 在同一侧；
5. **跨 harness 收集轨迹**：在一个系统里收集的轨迹会教模型「这个系统的约定」，换到 Claude Code 或 Codex 可能不泛化；
6. **平衡按 token，也要评测你正在改进的能力**，不要只在一个领域上 bench-max。

7.3 本章小结

训练沿记录轨迹、部署要自己走路，二者失配意味着 loss 不能替代评测。评测要先用过拟合小样本做 sanity check，再用 harness 跑可检查任务；读数字要对齐 split 与 harness，并跨 harness 收集数据以防「只会在一个系统里工作」。

8 SFT 与 RL 的交接：SFT 的收益在哪里

8.1 三条经验

课堂给出三条经验（以 DeepSeek R1.0 与 OpenThoughts 系工作为据）：第一，**完全不做 SFT、直接对 base model 做 RL，几乎带不动 base model**；第二，**只做 SFT 也远达不到 SFT+RL 的高度**；第三，最佳配方是从一个「**故意没有训到自身最高分、而是为后续 RL 训得更好**」的 checkpoint 开始 RL。

课堂用一张表展示「SFT 与 RL 的配比」：ColdSFT+RL 的 8B 模型在 SWE-bench Verified (100)、OT-TBLite、Terminal-Bench 2.0 上分别达到 35.7、16.0、13.5，平均 21.7；只做 SFT 的版本平均 15.1–15.9；完全不 SFT 直接 RL 只有 3.6；base 模型 2.7。课堂的总结句是：「**RL 的收益最大，当 SFT 模型是带着 RL 的目标选出来的**。」

8.2 没有更强的教师怎么办

课堂最后的问答讨论了一个很实际的问题：如果没有比学生更强的模型来生成轨迹，怎么办？课堂给了两条路：一是像 MAI 那样**自蒸馏**（用模型自己的推理轨迹）；二是像一些前沿实验室那样，把 SFT 压缩成「冷启动」，主要依靠 **RL** 提升能力。课堂的判断是：SFT 的天花板受教师能力限制，如果没有更好的教师，就必须更多依赖 RL。

8.3 要不要监督错误答案

另一个问答问到「是否要在错误答案上做监督」。课堂的回答是分两层的：**中间的错误动作值得监督**（模型要学「做错了怎么恢复」），但**最终结果必须正确**；也就是说，监督的是「错误 + 恢复 + 正确结果」的完整过程，而不是「做错并停在错处」。这和前面「成功轨迹里也可能有坏行为」的提醒互相呼应：数据筛选要保留恢复能力、剔除纯粹的坏习惯。

8.4 本章小结

SFT 提供通用 agent 能力的冷启动，RL 在它之上磨领域能力；「不 SFT 直接 RL」几乎无效，而「SFT 训到最高分再 RL」也不是最优——要给 RL 留空间。没有更强教师时靠自蒸馏或 RL；数据上要监督「错误 + 恢复」，但最终结果必须正确。

9 总结与延伸

本讲把 SFT 放回完整训练流水线中：它是 RL 的冷启动，负责把通用 agent 能力（工具调用、格式、长任务）写进权重。全讲可以压缩为六条：

1. 更新 agent 有三条路（上下文、外部工件、权重）；改权重换来快推理与彻底行为改变，代价是训练成本、黑盒与模型绑定。
2. 轨迹先被 chat template 渲染成一个 token 串，再用 0/1 mask 只让 assistant token 进交叉熵——SFT 本质是行为克隆。
3. mask 的细节决定学到什么：end-of-turn、reasoning 块、样本权重 vs token 权重，都需要显式控制；这些没有统一答案，要做消融。
4. 数据决定上限：来源（强模型/自己、是否过滤）、过滤（最小轮数等）、教师选择（最强 ≠ 最好教）、任务来源（4–8 个混合、增广优于上采样）。
5. 工程细节会静默毁掉训练：打包不截断、模板不许漂移、MoE 路由要平衡；公开技术报告与训练工件是最好的教材。
6. 训练 loss 不能替代评测：训练沿记录轨迹、部署要自己走路；先用 20 个样本过拟合做 sanity check，再对齐 harness 与 split。

一句话记住本讲

SFT 把「记录下来的轨迹」变成「权重里的行为倾向」；它决定 agent 的起点，但不决定终点——终点要靠 RL，而起点要专门为 RL 挑。

延伸思考：(1) 把 mask 当作一种「数据加权」来看，agent SFT 里所有关于 reasoning/tool call/observation 的取舍，都是在决定「让哪些 token 塑造模型」；(2) 教师选择与数据来源选择的收益，可能比调学习率大一个量级，这意味着 agent SFT 的核心工作更像是「数据工程 + 评测设计」，而不是「训练脚本调参」；(3) 模板漂移、打包截断这类问题与「评测口径」问题同源——它们都在问「训练时看到的世界，和部署时的世界是不是同一个」。下一讲进入强化学习基础，讨论如何用奖励而不是示范来继续提升 agent。